

Technical Spec: Visual Conversion Tracking Pixel

Overview

A lightweight, embeddable JavaScript pixel that, once placed on any page via a single global snippet, allows a user to visually select buttons or links and designate them as conversion events, or to define URL-based conversion rules (URL equals / URL contains). Designed to function like the Facebook Business Manager Event Setup Tool, but provider-agnostic and compatible with any public URL.

The system has three core parts:

1. A global tracking script (the pixel) deployed once per site.
 2. A visual selector overlay (the Event Setup Tool equivalent) for point-and-click event definition.
 3. A backend that stores event definitions, receives conversion events, and serves configuration back to the pixel.
-

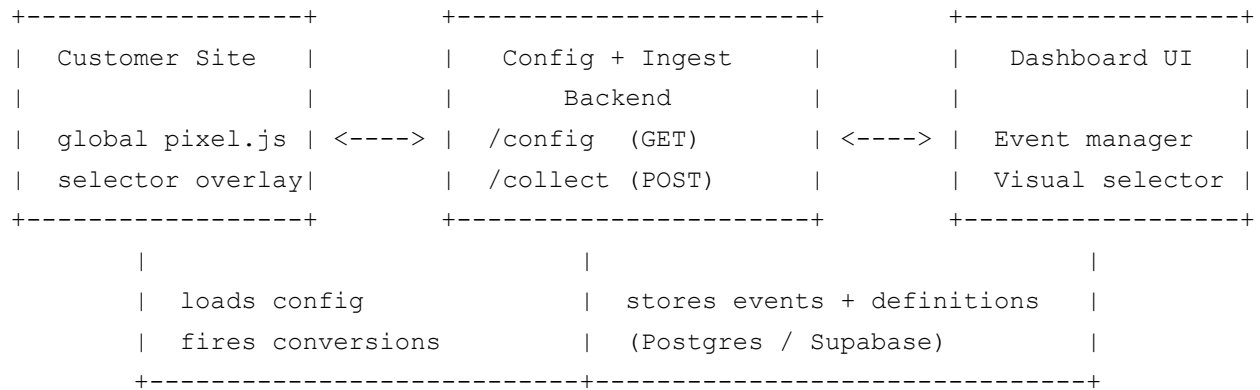
Goals

- Deploy once, configure forever. The site owner pastes one script tag and never touches code again.
- Visual event creation. Non-technical users select an element on their live page and label it a conversion.
- Two tracking modes: element-based (button or link clicks) and URL-based (pageview matching).
- Resilient selectors that survive minor DOM changes.
- Minimal performance footprint and no blocking of page render.
- Provider-agnostic, so conversions can be forwarded onward (server-side, webhook, or analytics destination) later.

Non-Goals (v1)

- Full-funnel attribution modeling.
- Cross-domain identity stitching.
- Server-side conversion forwarding to ad platforms (CAPI). This is a logical v2 and the data model should leave room for it.

Architecture



Components

1. Global pixel (`pixel.js`) The always-on script. On load it pulls the site's event configuration, attaches listeners, watches for URL changes, and fires conversion beacons. Also responsible for entering "selector mode" when triggered.

2. Selector overlay (`selector.js`) Loaded on demand (only when the user activates the Event Setup Tool). Renders a highlight overlay, captures the clicked element, computes a robust selector, and posts the new event definition to the backend.

3. Backend Serves per-site config, ingests conversion beacons, and persists event definitions. Endpoints: `/config`, `/collect`, plus authenticated CRUD for event management.

The Global Pixel Snippet

A single async snippet placed before the closing `</body>` (or in a tag manager). It self-bootstraps and exposes a small global API.

```

<!-- Conversion Pixel -->
<script>

```

```

(function(w,d,s,id){
  if (w.cpx) return;
  w.cpx = w.cpx || function(){ (w.cpx.q = w.cpx.q || []).push(arguments); };
  w.cpx.id = id;
  var js = d.createElement(s);
  js.async = true;
  js.src = "https://cdn.yourdomain.com/pixel.js?id=" + id;
  var f = d.getElementsByTagName(s)[0];
  f.parentNode.insertBefore(js, f);
})(window, document, "script", "SITE_PUBLIC_ID");
</script>
<!-- End Conversion Pixel -->

```

Notes:

- `SITE_PUBLIC_ID` is a non-secret public identifier scoped to one property. It is safe to expose in HTML.
- A command queue (`cpx.q`) buffers any calls made before `pixel.js` finishes loading, so nothing is lost on slow connections.
- Async load means zero render blocking.

Tracking Modes

Mode 1: Element-Based (Buttons and Links)

The user enters selector mode, clicks an element, and the tool records a durable selector for it. At runtime the pixel listens for clicks on matching elements and fires a conversion.

Selector strategy (priority order): The tool generates the most stable selector available, falling back gracefully:

1. `id` attribute, if present and unique (`#book-now`).
2. A stable `data-*` attribute (e.g. `data-cpx` , `data-testid` , `data-track`).
3. A semantic attribute combo: `tag + name` , `type` , `aria-label` , or trimmed inner text for buttons/links (e.g. `button` with text "Book a Call").
4. A scoped CSS path: a short path from the nearest stable ancestor, avoiding brittle full-document paths and auto-generated class hashes.

Each event definition should store **multiple matching signals**, not just one selector, so a match can be confirmed even if the primary selector drifts. Suggested matcher object:

```

{
  "type": "element",
  "matchers": {
    "id": "book-now",
    "dataAttr": { "name": "data-cpx", "value": "cta-hero" },
    "tag": "button",
    "text": "Book a Call",
    "href": null,
    "cssPath": "section.hero > div.cta > button",
    "ariaLabel": "Book a Call"
  },
  "matchMode": "any_strong"
}

```

Runtime matching at click time: Use a single delegated listener on `document` (capture phase) rather than binding to each element. On every click, walk up from `event.target` through ancestors and test each tracked definition against the element:

- **Strong match:** `id` or `data-*` matches. Fire immediately.
- **Medium match:** `tag + text` (case-insensitive, trimmed) or `tag + href`.
- **Weak match:** `cssPath` via `element.matches()`.

Delegation handles dynamically injected elements automatically and avoids re-binding on DOM changes.

Text and href normalization:

- Trim and collapse whitespace, lowercase for comparison.
- For links, compare both the visible text and a normalized `href` (strip trailing slash, optionally ignore query string per a rule flag).

Mode 2: URL-Based (Pageview Conversions)

For thank-you pages, confirmation routes, or any URL signaling success. Two operators in v1:

- **URL equals:** exact match against the normalized current URL.
- **URL contains:** substring match against the normalized current URL.

(Recommended to also stub **URL matches regex** in the data model for v2, even if hidden in the v1 UI.)

```
{
  "type": "url",
  "operator": "contains",      // "equals" | "contains" | "regex"
  "value": "/thank-you",
  "match": "path_and_query",  // "path" | "path_and_query" | "full_url"
  "caseSensitive": false
}
```

Normalization rules:

- Lowercase host and (optionally) path.
- Strip trailing slash.
- A per-rule flag controls whether query string and hash are included in the comparison.

SPA handling: Single-page apps do not trigger full page loads on navigation, so the pixel must detect soft route changes:

- Patch `history.pushState` and `history.replaceState`.
- Listen for `popstate` and `hashchange`.
- On any of these, re-run all URL rules against the new location.
- De-duplicate so the same URL conversion does not fire twice for one logical navigation (track last-fired URL per rule).

The Visual Selector Overlay (Event Setup Tool)

This is the point-and-click experience that mirrors Facebook's tool.

How a user enters selector mode: A few options, support more than one:

1. From the dashboard, the user enters their site URL. The site opens with a `?cpx_setup=1&token=...` parameter that the pixel detects and uses to launch selector mode.
2. A short-lived setup token authorizes the session so random visitors cannot open the editor.

Overlay behavior:

- Inject a full-viewport transparent layer that intercepts `mouseover` and `click`.
- On hover, draw a highlight box (outline plus subtle fill) around the element under the

cursor and show a small tooltip with the element tag and a snippet of its text.

- On click, freeze selection, prevent the element's default action (`preventDefault + stopPropagation`), and open a small inline panel:
 - Detected element summary (tag, text, generated selectors).
 - Field: event name (e.g. "Lead", "Book Call", "Purchase").
 - Optional value and currency fields.
 - Confirm / cancel.
- On confirm, compute the matcher object and POST it to the backend, then visually mark the element as tracked.

Element resolution helper:

- Clicking a child (an `<span>` or `<i>` icon inside a button) should resolve up to the nearest meaningful interactive ancestor (`a` , `button` , `[role=button]` , `input[type=submit]`).
- Show the user which element was actually selected and let them step up/down the DOM tree (a "select parent" control) if the resolution guessed wrong.

Live config preview: While in setup mode, already-tracked elements should be outlined in a distinct color so the user can see what is already configured.

Backend

Endpoints

GET /config?id=SITE_PUBLIC_ID Returns the active event definitions for a site. Cacheable at the edge with a short TTL (for example 60 seconds) plus a cache-busting version param so config updates propagate quickly.

```
{
  "siteId": "SITE_PUBLIC_ID",
  "version": "2026-05-23T19:00:00Z",
  "events": [
    { "id": "evt_1", "name": "Book Call", "type": "element", "matchers": { } },
    { "id": "evt_2", "name": "Lead", "type": "url", "operator": "contain"
  ]
}
```

POST /collect Receives a fired conversion. Should be sent via `navigator.sendBeacon()` when available (survives page unload), falling back to `fetch` with `keepalive: true`.

```
{
  "siteId": "SITE_PUBLIC_ID",
  "eventId": "evt_1",
  "eventName": "Book Call",
  "type": "element",
  "url": "https://example.com/pricing",
  "referrer": "https://google.com",
  "ts": 1748025600000,
  "anonId": "uuid-first-party-cookie",
  "session": "uuid-session",
  "value": null,
  "currency": null,
  "meta": { "elementText": "Book a Call", "elementTag": "button" }
}
```

Authenticated event CRUD (dashboard only, behind auth):

- **POST /events create**
- **GET /events?siteId= list**
- **PATCH /events/:id update** (rename, change value, toggle active)
- **DELETE /events/:id**

Data Model (illustrative)

```
sites
  id (public_id)      text  primary key
  owner_id            uuid
  domain              text
  created_at          timestampz

events
  id                  uuid  primary key
  site_id             text  references sites(id)
  name                text
  type                text  enum: element | url
  matchers            jsonb          -- element mode
  url_operator         text          -- url mode: equals | contains | regex
  url_value            text
  url_match_scope      text          -- path | path_and_query | full_url
  case_sensitive       boolean
```

value	numeric null
currency	text null
active	boolean default true
created_at	timestampz
updated_at	timestampz

conversions	
id	uuid primary key
site_id	text
event_id	uuid
url	text
referrer	text
anon_id	text
session_id	text
value	numeric null
currency	text null
meta	jsonb
occurred_at	timestampz

Identity and Sessions

- Set a first-party cookie (or `localStorage` fallback) with an `anonId` UUID on first load. This is the visitor identifier and stays first-party only.
- Generate a `sessionId` per session (30 minute inactivity rollover is a reasonable default).
- No third-party cookies. This keeps the pixel privacy-friendly and resilient to browser tracking restrictions.

Deduplication and Reliability

- **Element clicks:** optionally debounce so a rapid double-click does not double-fire. A per-(eventId, anonId) short window (for example 1 second) is usually enough.
- **URL conversions:** track the last URL that fired each URL rule per session to avoid duplicate fires on the same logical page.
- **Beacon delivery:** prefer `sendBeacon`. If it returns false or is unavailable, use `fetch(..., { keepalive: true })`. Queue and retry on transient failure where feasible.
- **Config failure:** if `/config` fails to load, the pixel should fail silently and not break the

host page. Cache the last good config in `localStorage` as a fallback.

Performance

- Keep `pixel.js` small (target well under 15KB gzipped). The selector overlay is a separate bundle loaded only in setup mode, so visitors never download it.
 - One delegated document listener for clicks rather than N element listeners.
 - Lazy-evaluate URL rules only on actual navigation events, not on a timer.
 - Serve from a CDN with edge caching.
-

Security and Abuse Considerations

- `SITE_PUBLIC_ID` is public and read-only. It cannot be used to read or write event definitions.
 - Selector/setup mode must require a short-lived authenticated token tied to the dashboard session, so a random visitor cannot open the editor or create events.
 - `/collect` should validate that the incoming `siteId` exists and rate-limit per IP/site to limit spam injection.
 - Optionally validate the `Origin / Referer` against the site's registered domain on `/collect`.
 - Sanitize any element text stored in `meta` (it is user-page content, treat as untrusted).
-

Edge Cases to Handle

- Element inside an iframe (out of scope for v1; document the limitation, since cross-origin iframes cannot be inspected).
- Element rendered after page load (handled by delegation, but URL rules should re-check on SPA route change).
- Same button appearing on multiple pages (matchers are site-wide by default; consider an optional per-event URL scope so an event only counts on certain pages).
- Buttons with identical text but different purposes (this is why `id / data-* /path` are

layered in, not just text).

- Links that navigate away immediately (the click fires before unload; `sendBeacon` is what makes this reliable).

Suggested Build Phases

Phase 1: Core pixel + URL tracking Global snippet, `/config`, `/collect`, first-party identity, URL equals/contains rules with SPA handling. URL rules are configurable from the dashboard by typing, no visual selector needed yet. Fastest path to a working conversion signal.

Phase 2: Visual element selector Selector overlay, matcher generation, element-click tracking with delegation, the Event Setup Tool UI, tracked-element preview.

Phase 3: Hardening + scope rules Dedup tuning, per-event URL scoping, config caching/fallback, rate limiting, origin validation, dashboard CRUD polish.

Phase 4 (v2): Forwarding Server-side forwarding/webhooks and a path toward platform Conversions APIs, using the data model hooks left in place.

Open Questions for the Team

1. Should an event be able to scope to specific pages (URL conditions) on top of an element match, or stay site-wide?
2. Do we want value/currency on conversions in v1, or defer to v2?
3. Where does this live relative to existing infrastructure: standalone, or as a module that shares identity/session with current tooling?
4. Confirmation strategy for "tracked correctly": do we want a live test mode where the user clicks the button and sees the event register in real time, like Facebook's Event Setup Tool does?